

Teil3: Aufbau einer Zwischendarstellung

Das Ziel der dritten Übung ist, die Grammatik mit semantischen Aktionen zu versehen, so dass für alle Anweisungen des Quelltextes eine entsprechende Zwischendarstellung (Drei-Adress-Code-Konstrukte) im Speicher aufgebaut wird, die als Basis für die Maschinen-Codeerzeugung dient. Die DAC-Konstrukte werden in Form einer *Tripel-Darstellung* (siehe Folienskript) gespeichert. Führen Sie dazu folgende Implementierungsschritte durch:

1. Als Schnittstelle für die Zwischencodegenerierung soll eine Klasse `DACGenerator` dienen. Sie stellt Methoden zur Verfügung, die DAC-Anweisungen erzeugen. Als Parameter dienen Operatoren und Symbole die entsprechend verknüpft werden und so im Speicher eine Abbildung der Anweisungen des Quelltextes darstellen.
2. Eine einzelne DAC-Anweisung wird durch eine Klasse `DACEntry` abgebildet und besteht aus einem Operator und zwei Argumenten, die wieder durch Symbole dargestellt werden.
3. Sprünge in einer Schleifen- oder Bedingungsanweisung können durch einen Verweis auf die entsprechende Zielanweisung abgebildet werden. Die Zielanweisung ist jene Anweisung, die abhängig von der Bedingung ausgeführt wird.
4. Erweiterung der ATG um semantische Aktionen die DAC-Anweisungen mit Hilfe des DAC-Generators erzeugen und in einem entsprechenden Container speichern.

Hinweis: Der `DACGenerator` wird in der ATG folgendermaßen inkludiert und deklariert:

```
1 #include "DACGenerator.h"
2
3 COMPILER MIEC
4
5     DACGenerator mDACGen;
6
7 CHARACTERS
8     ...
9 TOKENS
10    ...
```

Durch diese Deklaration wird der `DACGenerator` als Attribut in der Klasse `Parser` erzeugt, und kann direkt in den semantischen Aktionen der ATG verwendet werden und den DAC-Code entsprechend erzeugen.

Die Operatoren im `DACEntry` können durch folgende Enumeration abgebildet werden:

```
1 enum class OpKind {  
2     eAdd, eSubtract, eMultiply, eDivide, eIsEqual, eIsLessEqual, eIsGreaterEqual,  
3     eIsNotEqual, eIsLess, eIsGreater, eAssign, eJump, eIfJump, eIfFalseJump, ePrint,  
4     eExit  
5 };
```

Beispiel zur DAC-Erzeugung: folgender Quellcode wird in DAC übersetzt:

```
1 PROGRAM Test
2   BEGIN_VAR
3     n: Integer;
4     fac: Integer;
5     val: Integer;
6   END_VAR
7 BEGIN
8   WHILE fac <= n DO
9     val := val * fac;
10    fac := fac + 1;
11  END;
12 END
```

DAC:

```
1 L1: ifFalse fac <= n GOTO L2
2   t1 = val * fac
3   val = t1
4   t2 = fac + 1
5   fac = t2
6   GOTO L1
7 L2: EXIT
```